Library Management System

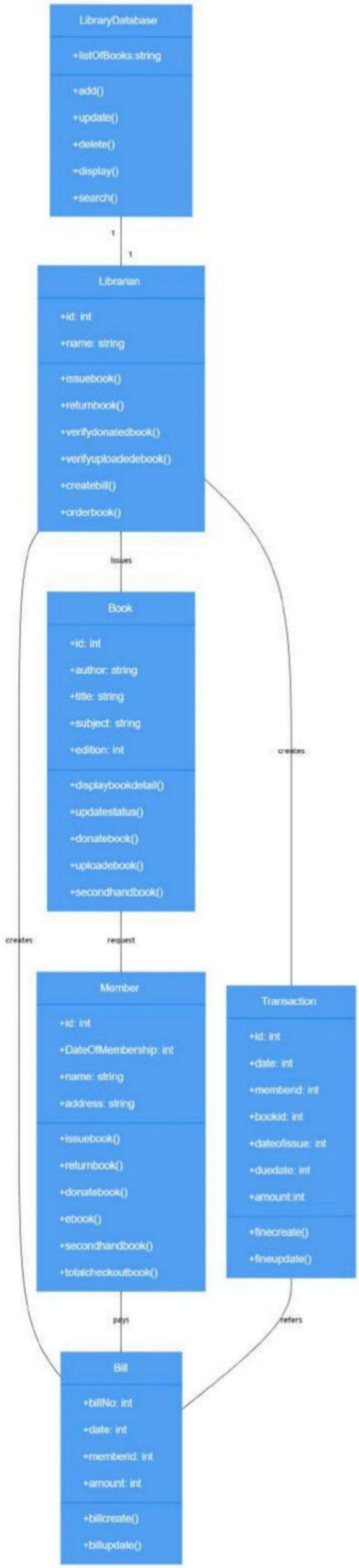
LibraLink

Redefining Library Access for the Digital Age.

P A Aparna, S3 CSE-B, 53
Aparna K P, S3 CSE-B, 12
Naheedha K A, S3 CSE-B, 42
Nandana P, S3 CSE-B, 44

Use Case Diagram

Class Diagram



ABSTRACT

In today's rapidly evolving educational landscape, efficient management of library resources is essential to support both academic and self-learning pursuits. ***LibraLink*** *is an intelligent and user-friendly* ***Library Management System*** *designed to automate and streamline all major library functions—from cataloging and issuing books to handling donations, returns, and second-hand book exchanges.*

The system is divided into two main modules: the ***Admin Dashboard*** *and the* ***Student Dashboard****. The* ***Admin Module*** *allows librarians to manage the collection of books, track issues and returns, update inventory, and approve second-hand book listings. The* ***Student Module*** *enables users to search for books, borrow and return them, request new titles, donate books, or list their own used books for exchange or resale within the campus library network.*

One of LibraLink's standout features is its ***Second-Hand Book Exchange****, which allows students to list used books they wish to sell or donate. Other students can browse available listings, making the system a* ***sustainable and cost-effective resource-sharing platform****. This feature not only promotes reusability and accessibility but also supports an eco-friendly approach to reading and learning.*

*Built using **Java, MySQL, and DAO (Data Access Object) architecture**, LibraLink ensures secure data handling, modular design, and smooth scalability. The system automatically tracks due dates (14 days from issue), manages availability updates, and records all transactions to prevent duplication or loss of data.*

*By combining automation with a community-driven exchange system, LibraLink bridges the gap between traditional libraries and modern digital ecosystems. It fosters **collaboration, sustainability, and digital empowerment**—truly living up to its name as the link between **knowledge, people, and purpose**.*

System Modules

• Authentication – Handles secure login for Admin and Student users.

• Book Management – Admin can add, update, and delete physical books.

• Search & Borrow – Students can search and borrow books using Title, Author, or ISBN. • Return & Fine Calculation – Calculates fines for overdue returns automatically.

• E-Book Management – Allows upload and access of digital books.

• *Book Donation – Students can donate books, with admin approval.*

• *Second-Hand Book Listing – Enables listing of old books for exchange or sale.*

• *Dashboard – Provides dedicated panels for Admin and Student operations.*

System Architecture

1. Overview:

The LibraLink system follows a ***3-tier architecture*** *model:*

- ***Presentation Layer (Frontend / UI):***
  Java Swing GUI forms such as `AdminDashboard`*,* `StudentDashboard`*,* `AddEbookUI`*,* `SearchBookUI`*, and* `IssueBookUI`*.*
  - *Handles user interactions and displays data.*
  - *Forwards user actions to backend logic.*
- ***Application Layer (Business Logic):***
  Java DAO (Data Access Object) classes handle the logic of borrowing, returning, issuing, and managing books.

- *Example classes: `BookDAO`, `BorrowDAO`, etc.*
- *Applies rules like “14 days due date” or “availability check”.*

- ***Data Layer (Database):***
  MySQL Database stores all persistent data.
  - *Tables: `books`, `ebooks`, `users`, `borrowed_books`, `second_hand_books`.*
  - *Ensures data integrity and relationships (foreign keys between users and books).*

2. Component Interaction:

- *UI sends requests → DAO → DB → Response back to UI.*
- *Example: Admin issues a book*
  - *`IssueBookUI` → `BorrowDAO.adminIssueBook()` → updates `borrowed_books` & `books` tables → success message on UI.*

Workflow Example: Book Issue & Return

Step 1 – Admin Issues a Book:

1. *Admin opens **IssueBookUI**.*
2. *Enters `user_id` and `book_id`.*
3. *System verifies if the book is available (`available > 0`).*
4. *`BorrowDAO.adminIssueBook()` inserts record in `borrowed_books`.*
5. *Due date = `borrow_date + 14 days`.*
6. *`books.available` decremented by 1.*
7. *Confirmation shown: "Book issued successfully."*

Step 2 – Student Returns a Book:

1. *Student logs into **StudentDashboard**.*
2. *Selects "Return Book" → enters `book_id`.*
3. *System checks if the record exists in `borrowed_books` for that user and `returned =`*

0.

4. *If yes → sets `returned = 1`, updates `return_date = today`.*

5. *`books.available` incremented by 1.*

6. *Confirmation message: "Book returned successfully."*

Second-Hand Books Feature

- *A new table `second_hand_books` allows students to list, donate, or request used books.*

- *Fields: `book_id`, `title`, `author`, `condition`, `donor_user_id`, `status`.*

- *DAO methods handle uploads, availability, and purchase requests.*

- *Encourages resource sharing and sustainability.*

Database Design

The system uses MySQL as the primary database. Major tables include users, books, borrowed_books,

donated_books, ebooks, secondhand_books, and fines. Each table is relationally linked through primary and foreign keys ensuring data consistency.

Tables:

1. `users` – *(user_id, username, password, role)*
2. `books` – *(book_id, title, author, publisher, edition, isbn, quantity, added_date)*
3. `ebooks` – *(ebook_id, title, author, file_path, isbn)*
4. `borrowed_books` – *(borrow_id, user_id, book_id, borrow_date, return_date, fine)*
5. `donated_books` – *(donate_id, title, author, publisher, edition, book_condition, isbn, uploaded_by, photo_path, status, upload_date)*

Prototype Creation

The prototype of the LibraLink Library Management System was developed through an iterative design and testing process to validate the system's user interface, data flow, and key functionalities before final implementation.

*The **UI Mockups** were created using **Java Swing components** such as `JFrame`, `JPanel`, `JButton`, `JLabel`, and `JTable`. These mockups provided the initial layout and interaction model for both **admin** and **student** users, later refined based on functional testing and usability feedback.*

The prototype included the following major modules and forms:

- ***LoginUI:** Handles authentication for both admins and students. It validates credentials and redirects users to the appropriate dashboard.*

- ***AdminDashboard:** Allows administrators to manage books, issue or return physical copies, add eBooks, and monitor system activity. It also includes the IssueBookUI for issuing books to students and tracking due dates.*

- ***StudentDashboard:** Enables students to search, borrow, or return books, access eBooks, and view their borrowing history.*

- ***AddBookUI:** Designed for administrators to add new book entries to the system database. It supports fields like title, author, category, publisher, and available quantity.*

- ***SearchBookUI:*** *Allows users to search books by title, author, or ISBN. Integrated with real-time database queries through the `BookDAO` class.*

- ***SecondHandBookUI (Prototype Addition):*** *Provides students an interface to upload details of second-hand books for exchange or resale, connecting peers within the institution.*

Each prototype screen was functionally linked to backend DAO classes (`BookDAO`, `BorrowDAO`, `UserDAO`), which handle database operations using MySQL through JDBC.

The development followed a ***stepwise refinement*** *approach:*

1. *Initial static mockups were created to visualize the layout.*

2. *Core functionalities like Search, Borrow, and Return were implemented and tested.*

3. *Inter-module connections (Admin ↔ Student ↔ Database) were validated.*

4. *Feedback-based adjustments were made to improve layout alignment, button placement, and*

error handling messages.

*Through this prototype, all key workflows—such as **issuing a book by admin**, **borrowing by students**, and **second-hand book listing**—were successfully simulated.*
This ensured that the design was user-friendly, consistent, and aligned with the overall objectives of LibraLink.

Source Code Development — LibraLink

*This document describes the source code development approach for **LibraLink** (Library Management System). It covers overall structure, packages, key classes and methods, database mapping, important code snippets, build/run instructions, testing notes, and recommended improvements. Use this as the authoritative implementation guide when organizing, extending, or submitting the project.*

---

1. Project Overview

*LibraLink is implemented in **Java** with a Swing-based UI and a **MySQL** backend accessed through **JDBC**. The project follows a simple layered architecture:*

- ***UI (presentation)*** *— `ui` package (Swing frames & dialogs)*
- ***Business / Data Access*** *— `dao` package (DAO classes)*
- ***Models*** *— `model` package (POJOs)*
- ***DB Utilities*** *— db package (DBConnection)*

The design emphasizes modular DAOs, single-responsibility UIs, and transactional DB operations for borrow/return flows.

---

2. Directory / Package Layout

src/

db/

DBConnection.java

model/

User.java

Book.java

Ebook.java

DonatedBook.java

SecondHandBook.java

dao/

BookDAO.java

BorrowDAO.java

EbookDAO.java

DonationDAO.java

UserDAO.java

ui/

LoginUI.java

AdminDashboard.java

StudentDashboard.java

SearchBookUI.java

IssueBookUI.java

AddBookUI.java

AddEbookUI.java

DonateBookUI.java

SecondHandBookUI.java

---

3. Database Schema (essential tables)

- *users(user_id PK, username, password, role)*
- *books(book_id PK, title, author, category, available INT, publisher, edition, isbn, quantity)*
- *borrowed_books(borrow_id PK, user_id FK, book_id FK, borrow_date, return_date, returned BOOLEAN)*
- *ebooks(ebook_id PK, title, subject, author, file_path, uploaded_by, status)*
- *donated_books(donate_id PK, title, author, uploaded_by, status, upload_date, photo_path, book_condition)*
- *second_hand_books(id PK, title, author, condition, price, owner_user_id, status)*

---

4. Core Model Classes

Example Book constructor (match DAO usage):

```
public class Book {
  private int bookId;
  private String title, author, category;
  private boolean available;
  private String publisher, edition, isbn;
  private int quantity;
  // getters, setters, and a constructor:
  public Book(int bookId, String title, String author, String category,
              boolean available, String publisher, String edition, String isbn, int quantity) { ... }
}
```

`User` contains `userId`, `username`, `password`, optionally `role`.

---

5. Key DAO Responsibilities & Patterns

- ***DBConnection.getConnection()*** *— central connection provider (use try-with-resources in DAOs).*

- ***Transactions*** *— Borrow/Return operations use transactions (setAutoCommit(false), commit/rollback) to maintain integrity.*

- ***PreparedStatement*** *— always use prepared statements to avoid SQL injection.*

- ***Exception handling*** *— catch SQL/checked exceptions, log stack traces, return boolean success/failure.*

6.Important DAO methods:

- *`BookDAO.searchBooks(String keyword)` → returns `List<Book>`*

- *`BorrowDAO.borrowBook(int userId, int bookId)` → checks availability, inserts borrow record, decrements `books.available`*

- *`BorrowDAO.returnBook(int userId, int bookId)` → finds active borrow record for that user+book, marks returned, increments availability*

- *`BorrowDAO.adminIssueBook(int userId, int bookId)` → inserts borrow with `return_date = borrow_date + 14 days`*

- *`EbookDAO.addEbook(Ebook ebook)` and `getApprovedEbooks(String keyword)`*

- *`DonationDAO.saveDonation(DonatedBook book)` and `getPendingDonations()`*

---

7. Build & Run Instructions

Compile project (Windows PowerShell example):

javac -cp ".;lib/mysql-connector-j-9.4.0.jar" -d .
(Get-ChildItem -Recurse src.java | ForEach-Object { $_.FullName })*

Run main UI (example):

java -cp ".;lib/mysql-connector-j-9.4.0.jar" ui.LoginUI

Ensure `lib/mysql-connector-j-9.4.0.jar` is present and MySQL credentials in `DBConnection` are correct.

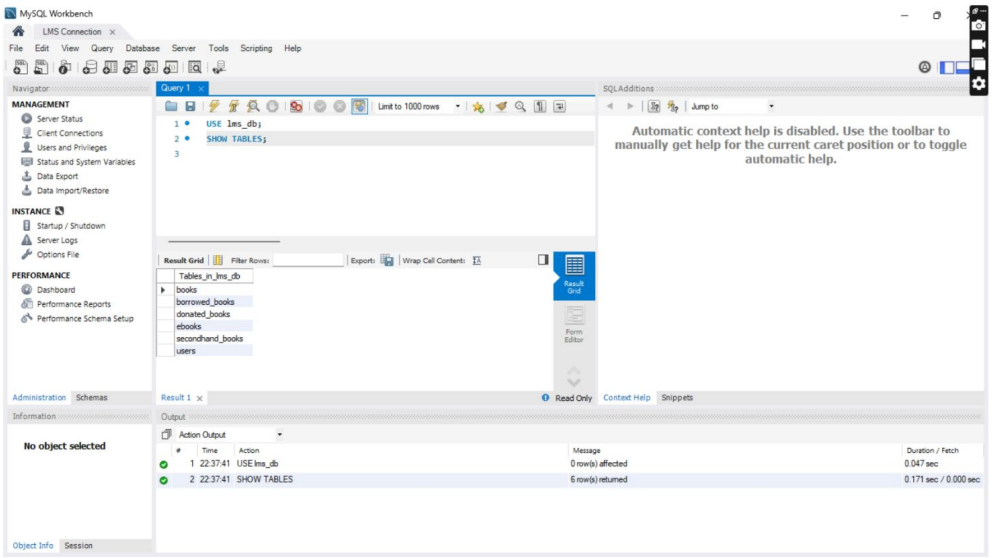
---

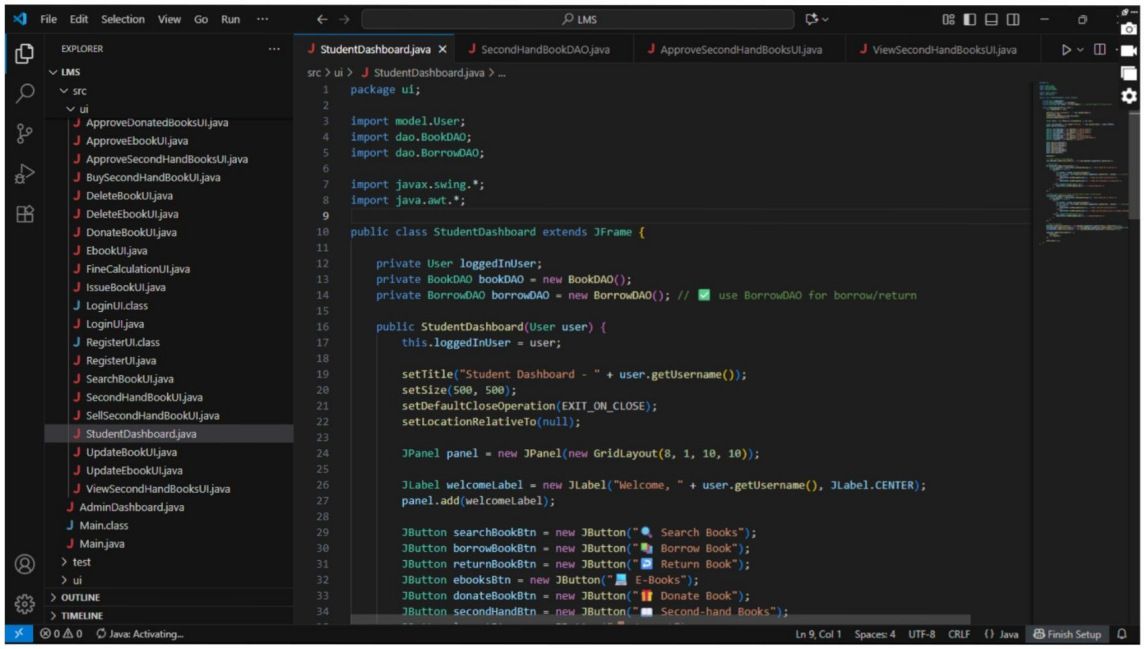
8. Testing & Validation

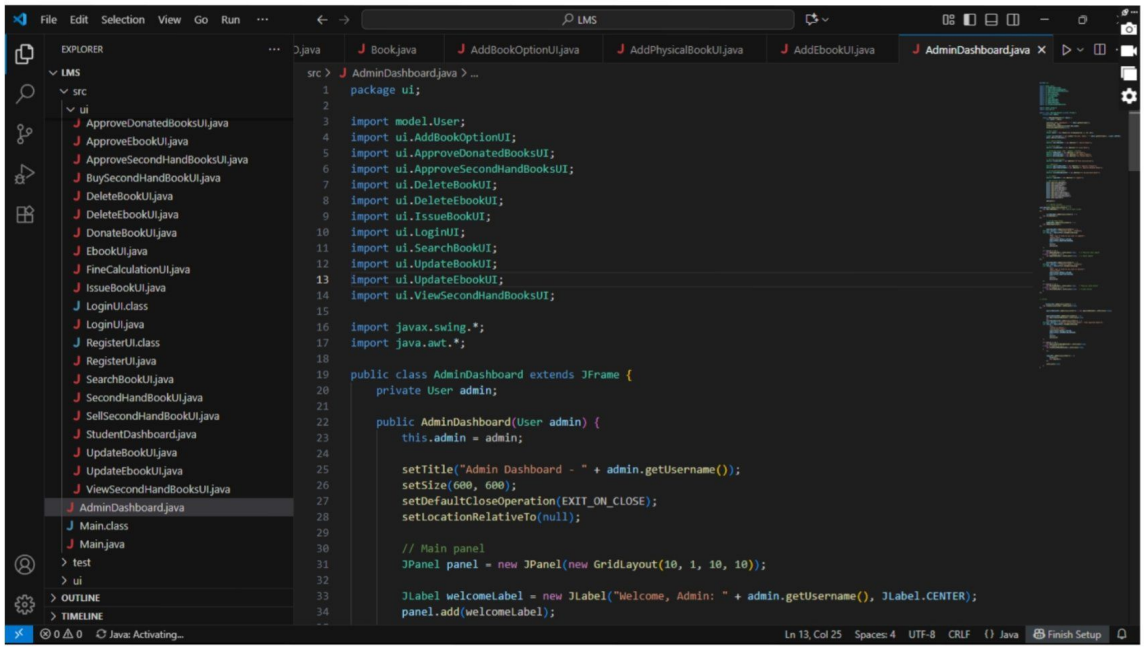
- *Unit-test DAOs with a test database (or use transaction rollback after tests).*
- *Manual UI tests: borrow and return cycles, admin issue, search, add eBook, donate flow, second-hand listing.*
- *Edge cases: borrow when `available=0`, return when already returned, invalid IDs.*

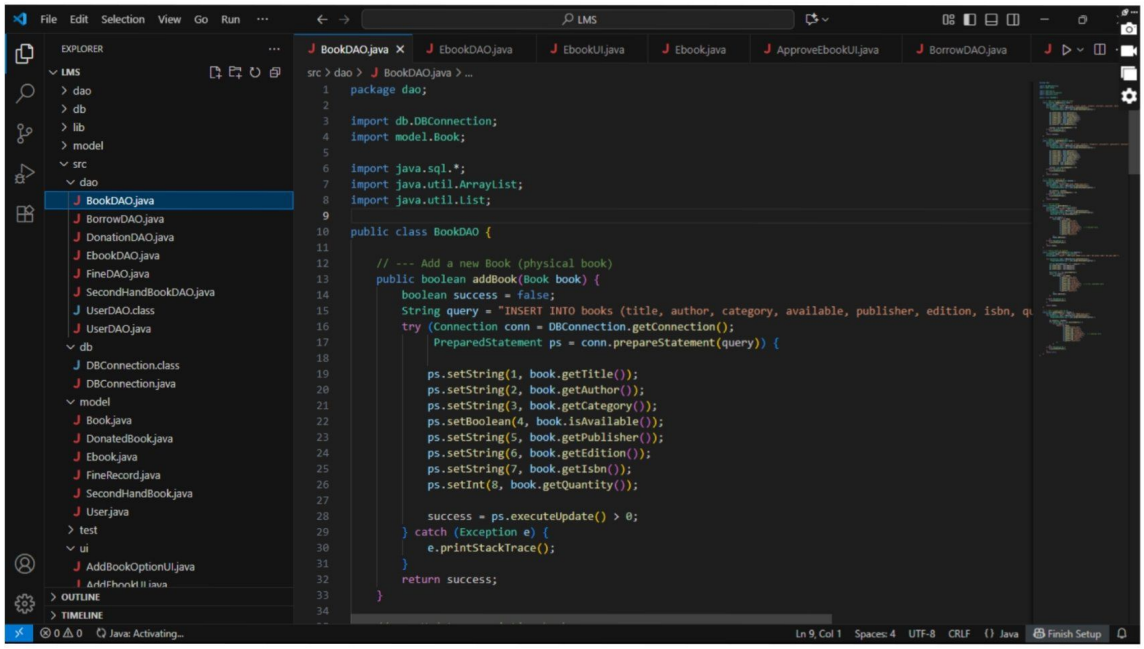
Here are the screen shots of the project:

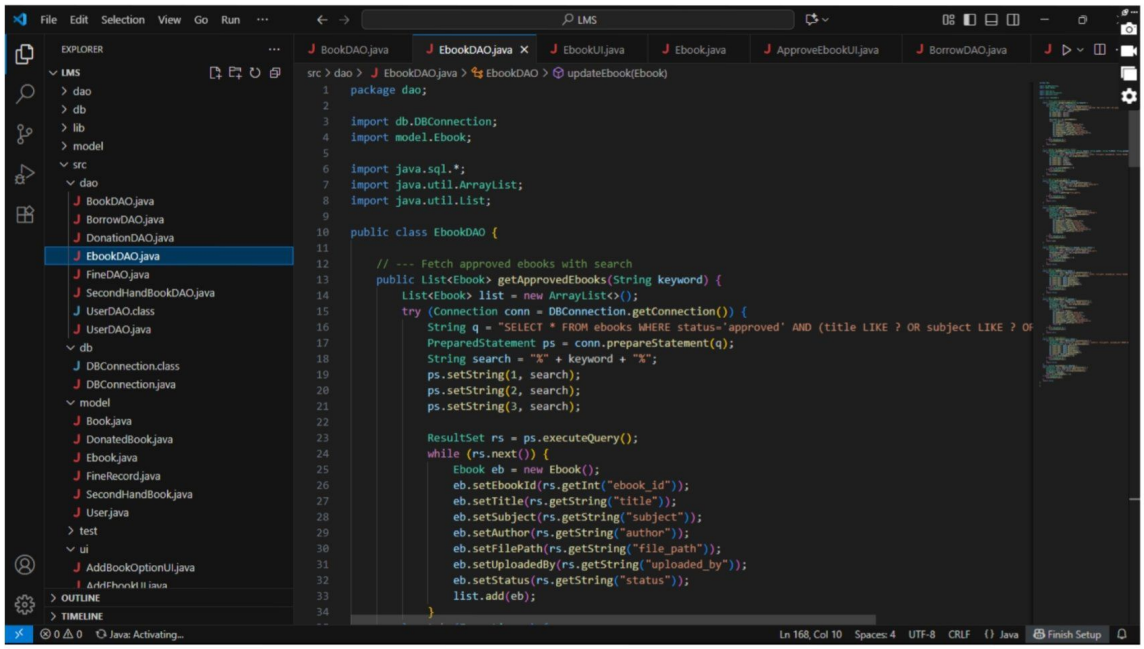
Database: SQL



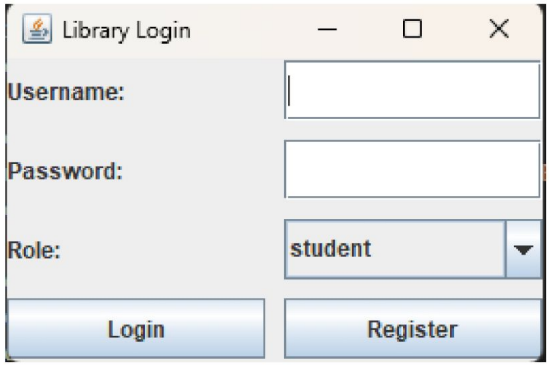

File Edit Selection View Go Run
LMS
EXPLORER
LMS
src
ui
ApproveDonatedBooksUI.java
ApproveEbookUI.java
ApproveSecondHandBooksUI.java
BuySecondHandBookUI.java
DeleteBookUI.java
DeleteEbookUI.java
DonateBookUI.java
EbookUI.java
FineCalculationUI.java
IssueBookUI.java
LoginUI.class
LoginUI.java
RegisterUI.class
RegisterUI.java
SearchBookUI.java
SecondHandBookUI.java
SellSecondHandBookUI.java
StudentDashboard.java
UpdateBookUI.java
UpdateEbookUI.java
ViewSecondHandBooksUI.java
AdminDashboard.java
Main.class
Main.java
test
ui
OUTLINE
TIMELINE
StudentDashboard.java
SecondHandBookDAO.java
ApproveSecondHandBooksUI.java
ViewSecondHandBooksUI.java
src > ui > StudentDashboard.java > ...
package ui;
import model.User;
import dao.BookDAO;
import dao.BorrowDAO;
import javax.swing.*;
import java.awt.*;
public class StudentDashboard extends JFrame {
private User loggedInUser;
private BookDAO bookDAO = new BookDAO();
private BorrowDAO borrowDAO = new BorrowDAO(); // ✅ use BorrowDAO for borrow/return
public StudentDashboard(User user) {
this.loggedInUser = user;
setTitle("Student Dashboard - " + user.getUsername());
setSize(500, 500);
setDefaultCloseOperation(EXIT_ON_CLOSE);
setLocationRelativeTo(null);
JPanel panel = new JPanel(new GridLayout(8, 1, 10, 10));
JLabel welcomeLabel = new JLabel("Welcome, " + user.getUsername(), JLabel.CENTER);
panel.add(welcomeLabel);
JButton searchBookBtn = new JButton("🔍 Search Books");
JButton borrowBookBtn = new JButton("📚 Borrow Book");
JButton returnBookBtn = new JButton("↩️ Return Book");
JButton ebooksBtn = new JButton("💻 E-Books");
JButton donateBookBtn = new JButton("🎁 Donate Book");
JButton secondHandBtn = new JButton("📖 Second-hand Books");
Ln 9, Col 1 Spaces: 4 UTF-8 CRLF Java Finish Setup
Java: Activating...


File Edit Selection View Go Run
LMS
EXPLORER
LMS
src
ui
ApproveDonatedBooksUI.java
ApproveEbookUI.java
ApproveSecondHandBooksUI.java
BuySecondHandBookUI.java
DeleteBookUI.java
DeleteEbookUI.java
DonateBookUI.java
EbookUI.java
FineCalculationUI.java
IssueBookUI.java
LoginUI.class
LoginUI.java
RegisterUI.class
RegisterUI.java
SearchBookUI.java
SecondHandBookUI.java
SellSecondHandBookUI.java
StudentDashboard.java
UpdateBookUI.java
UpdateEbookUI.java
ViewSecondHandBooksUI.java
AdminDashboard.java
Main.class
Main.java
test
ui
OUTLINE
TIMELINE
Book.java
AddBookOptionUI.java
AddPhysicalBookUI.java
AddEbookUI.java
AdminDashboard.java
src > AdminDashboard.java > ...
package ui;
import model.User;
import ui.AddBookOptionUI;
import ui.ApproveDonatedBooksUI;
import ui.ApproveSecondHandBooksUI;
import ui.DeleteBookUI;
import ui.DeleteEbookUI;
import ui.IssueBookUI;
import ui.LoginUI;
import ui.SearchBookUI;
import ui.UpdateBookUI;
import ui.UpdateEbookUI;
import ui.ViewSecondHandBooksUI;
import javax.swing.*;
import java.awt.*;
public class AdminDashboard extends JFrame {
private User admin;
public AdminDashboard(User admin) {
this.admin = admin;
setTitle("Admin Dashboard - " + admin.getUsername());
setSize(600, 600);
setDefaultCloseOperation(EXIT_ON_CLOSE);
setLocationRelativeTo(null);
// Main panel
JPanel panel = new JPanel(new GridLayout(10, 1, 10, 10));
JLabel welcomeLabel = new JLabel("Welcome, Admin: " + admin.getUsername(), JLabel.CENTER);
panel.add(welcomeLabel);
Ln 13, Col 25 Spaces: 4 UTF-8 CRLF Java Finish Setup
Java: Activating...

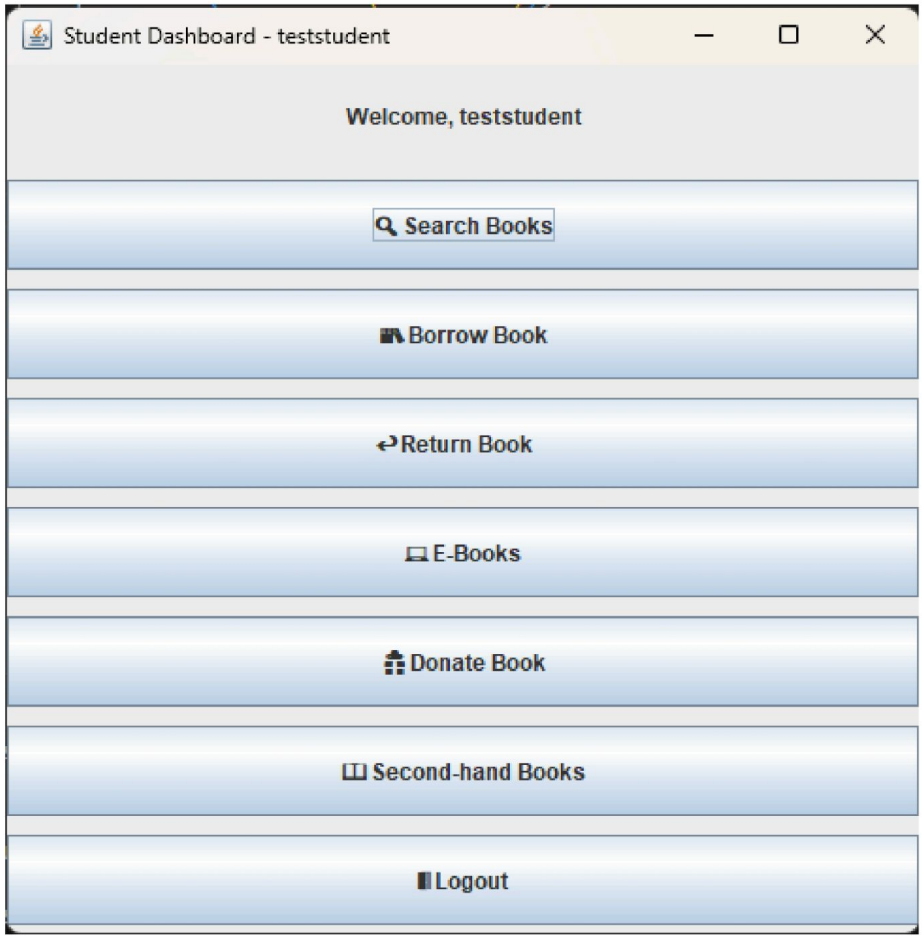

File Edit Selection View Go Run
LMS
EXPLORER
LMS
dao
db
lib
model
src
dao
BookDAO.java
BorrowDAO.java
DonationDAO.java
EbookDAO.java
FineDAO.java
SecondHandBookDAO.java
UserDAO.class
UserDAO.java
db
DBConnection.class
DBConnection.java
model
Book.java
DonatedBook.java
Ebook.java
FineRecord.java
SecondHandBook.java
User.java
test
ui
AddBookOptionUI.java
OUTLINE
TIMELINE
BookDAO.java
EbookDAO.java
EbookUI.java
Ebook.java
ApproveEbookUI.java
BorrowDAO.java
src > dao > BookDAO.java > ...
package dao;
import db.DBConnection;
import model.Book;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
public class BookDAO {
// --- Add a new Book (physical book)
public boolean addBook(Book book) {
boolean success = false;
String query = "INSERT INTO books (title, author, category, available, publisher, edition, isbn, q
try (Connection conn = DBConnection.getConnection();
PreparedStatement ps = conn.prepareStatement(query)) {
ps.setString(1, book.getTitle());
ps.setString(2, book.getAuthor());
ps.setString(3, book.getCategory());
ps.setBoolean(4, book.isAvailable());
ps.setString(5, book.getPublisher());
ps.setString(6, book.getEdition());
ps.setString(7, book.getIsbn());
ps.setInt(8, book.getQuantity());
success = ps.executeUpdate() > 0;
} catch (Exception e) {
e.printStackTrace();
}
return success;
}
Java: Activating...
Ln 9, Col 1
Spaces: 4
UTF-8
CRLF
Java
Finish Setup


File Edit Selection View Go Run
LMS
EXPLORER
LMS
dao
db
lib
model
src
dao
BookDAO.java
BorrowDAO.java
DonationDAO.java
EbookDAO.java
FineDAO.java
SecondHandBookDAO.java
UserDAO.class
UserDAO.java
db
DBConnection.class
DBConnection.java
model
Book.java
DonatedBook.java
Ebook.java
FineRecord.java
SecondHandBook.java
User.java
test
ui
AddBookOptionUI.java
OUTLINE
TIMELINE
BookDAO.java
EbookDAO.java
EbookUI.java
Ebook.java
ApproveEbookUI.java
BorrowDAO.java
src > dao > EbookDAO.java > EbookDAO > updateEbook(Ebook)
package dao;
import db.DBConnection;
import model.Ebook;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
public class EbookDAO {
// --- Fetch approved ebooks with search
public List<Ebook> getApprovedEbooks(String keyword) {
List<Ebook> list = new ArrayList<>();
try (Connection conn = DBConnection.getConnection()) {
String q = "SELECT * FROM ebooks WHERE status='approved' AND (title LIKE ? OR subject LIKE ? OR
PreparedStatement ps = conn.prepareStatement(q);
String search = "%" + keyword + "%";
ps.setString(1, search);
ps.setString(2, search);
ps.setString(3, search);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
Ebook eb = new Ebook();
eb.setEbookId(rs.getInt("ebook_id"));
eb.setTitle(rs.getString("title"));
eb.setSubject(rs.getString("subject"));
eb.setAuthor(rs.getString("author"));
eb.setFilePath(rs.getString("file_path"));
eb.setUploadedBy(rs.getString("uploaded_by"));
eb.setStatus(rs.getString("status"));
list.add(eb);
}
Java: Activating...
Ln 168, Col 10
Spaces: 4
UTF-8
CRLF
Java
Finish Setup

Login page:



Student dashboard:



Admin dashboard: